[REVISED PUBLIC] Connect on Z Quickstart



Confluent Connect on Z with IBM MQ Quickstart

This guide outlines setting up Confluent Connect on z/OS to integrate with IBM MQ using the source and sink connectors in bindings mode, leveraging zIIP.

1. **Prerequisites**

- **Environment:**
  - z/OS v2r4 or newer.
  - IBM Java 10 or newer (know the install path - `JAVAPATH`).
  - zIIP access enabled.
- **Access & User Setup:**
  - SSH/SFTP access to the z/OS system (ports 22/443 typically).
  - A USS (Unix System Services) user account with read/write/execute permissions in its home directory.
  - USS user requires ~5GB disk space and sufficient RAM for a 2GB+ Java Heap.
  - USS user needs permissions to run shell commands (bash/sh - Bash is better as we can run scripts unmodified).
- **System Details Needed:**
  - Kafka Cluster: Bootstrap server address, API Key, and Secret.
  - IBM MQ: Queue Manager name(s), target Queue name(s)
  - IBM MQ Install Path: Location of MQ Java libraries (needed for `LIBPATH` / `CLASSPATH`).

- **Network Ports:**
  - Ports required by Kafka Connect REST API (default 8083 for standalone, 8082 for distributed) need to be open on z/OS, or alternative ports configured (`rest.port` property).
  - Outbound over 9092

2. **Software Setup & File Preparation**

- **Get IBM MQ Libraries:**
  - Locate `libmqjbnd.so` and `libmqjbnd64.so` in your MQ installation's `java/lib` directory (e.g., `/usr/lpp/mqm/V9R2M0/java/lib` or `/opt/mqm/java`).
  - **Set LIBPATH:** Ensure the directory containing these `.so` files is included in the `LIBPATH` environment variable *before* starting Connect. Set this at the system/user environment level, not just in the Connect start script.

```
Sh
```

```
export LIBPATH=$LIBPATH:/path/to/mq/java/lib
```

  - **OR** set the following environment variables:
    - The `STEPLIB` used at run time must contain the IBM MQ `SCSQAUTH` and `SCSQANLE` libraries. Specify this library in the startup JCL, or specify it by using the `.profile` file.
      From UNIX and Linux System Services, you can add these using a line in your `.profile` file as shown in the following code snippet, replacing `thlqual` with the high-level data set qualifier that you chose when installing IBM MQ:
    - ```
      export STEPLIB=thlqual.SCSQAUTH:thlqual.SCSQANLE:$STEPLIB
      ```
    - The connector needs to load a native library. Set `LIBPATH` to include the following directory of your MQ installation:
    - ```
      <path_to_MQ_installation>/mqm/<MQ_version>/java/lib
      ```
  - Locate or obtain `com.ibm.mq.allclient.jar` from IBM.
  - **Potential CLASSPATH Issue:** Due to changes in MQ v9.2+, `com.ibm.mq.allclient.jar` might not be sufficient. If you encounter `ClassNotFoundException` for `MQConnectionFactory` or similar,

explicitly add specific JARs from the MQ `java/lib` directory to your `CLASSPATH` environment variable. *Do not* prepend with `$CLASSPATH:` to avoid duplicates:

```
Sh
```

```
export
CLASSPATH=/usr/lpp/mqm/java/lib/com.ibm.mqjms.jar:/usr/lpp/mqm/java/lib/com.ibm.mq.jmqi.
jar:/usr/lpp/mqm/java/lib/com.ibm.mq.allclient.jar
# Add other required MQ jars if necessary
```

- **Download Confluent Software:**
  - Download the "IBM MQ Source and Sink connectors for z/OS" from Confluent Hub. Extract the contents into a known location within your USS home directory (e.g., `/u/connectuser/connectors/mq-source`, `/u/connectuser/connectors/mq-sink`). You'll need these paths for `plugin.path` later.
  - Download the "Connect on z/OS" tarball.
- **Transfer and Unpack on z/OS:**
  - **Unpack Locally First:** The `tar.gz` file from Confluent may not unpack correctly with older `tar` versions on USS. It's best to unpack it on your local machine, then repackage it without extended attributes, and then transfer the repackaged `.tar` file:

```
Bash
```

```
# On local machine
tar -xzvf confluent-connect-<version>-zOS.tar.gz
tar --no-xattrs -cf confluent-connect-<version>-zOS.tar confluent-connect-<version>-zOS
# Transfer the new .tar file via SFTP/SCP

```

  - Transfer the repackaged Connect `.tar` file and the extracted connector directories to your USS home directory. Use binary transfer mode (e.g., `sftp`).
  - Unpack the Connect `.tar` file on USS: `tar -xf confluent-connect-<version>-zOS.tar`
- **EBCDIC Conversion (Crucial):** Configuration files must be in EBCDIC encoding. Files in `confluent-connect-<version>-zOS/etc/kafka` need conversion and the scripts in `/bin`. Other files (libs, share, etc.) should remain binary/ASCII.

- **Method 1 (Recommended if transferring individual files):** Use `scp` in ASCII mode for `/etc/kafka` and `/bin` files and `sftp` (binary mode) for everything else.
- **Method 2 (If transferring archive first):** Use `iconv` or `dd` on USS after unpacking to convert files *within* the `/etc/kafka` directory:

```
Sh
```

```
# Determine codset
locale -k codeset

# Example using iconv
cd /path/to/confluent/etc/kafka
iconv -f ISO8859-1 -t IBM-1047 ./connect-standalone.properties > ./connect-standalone.properties.ebcdic
mv ./connect-standalone.properties.ebcdic ./connect-standalone.properties

# Repeat for other .properties files you create/modify
```

3. **Configuration (Standalone Mode)**

Navigate to the `etc/kafka` directory within your unpacked Connect installation.

- `connect-standalone.properties`: Configures the Connect framework. Edit this file (ensure it's EBCDIC).

```
Properties
```

```
# Kafka Cluster Connection (Update with your details)
bootstrap.servers=YOUR_KAFKA_BOOTSTRAP:9092
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="YOUR_KAFKA_API_KEY" password="YOUR_KAFKA_API_SECRET";
ssl.endpoint.identification.algorithm=https

# Producer settings (used by source connectors) - inherits most settings above if connecting to same cluster
# producer.sasl.jaas.config=... (if different from worker)

# Consumer settings (used by sink connectors) - inherits most settings above if connecting to same cluster
# consumer.sasl.jaas.config=... (if different from worker)

# Converters (adjust if using Avro, JSON Schema, etc.)
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.storage.StringConverter

# Standalone specifics
offset.storage.file.filename=/tmp/connect.offsets  # Ensure USS user can write here
offset.flush.interval.ms=10000

# Plugin Path (Update with your actual paths)
```

```
plugin.path=/u/connectuser/connectors/mq-source,/u/connectuser/connectors/mq-sink

# REST API Port (Default 8083 for standalone)
# rest.port=8083

```

- `mq-source.properties`: Create this file (ensure it's EBCDIC) to configure reading from MQ.

```
Properties
```

```
name=my-mq-source-connector
connector.class=io.confluent.connect.ibm.mq.IbmMQSourceConnector
tasks.max=1 # Adjust as needed

# MQ Connection (Bindings Mode)
mq.queue.manager=YOUR_QMGR_NAME
# mq.hostname= (Leave blank or remove for bindings)
# mq.channel= (Leave blank or remove for bindings)
mq.transport.type=bindings
jms.destination.name=YOUR_MQ_SOURCE_QUEUE_NAME
jms.destination.type=queue
# mq.user.name= (If needed)
# mq.password= (If needed)

# Kafka Destination
kafka.topic=YOUR_KAFKA_DESTINATION_TOPIC

# Converters (usually inherit from worker, but can override)
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.storage.StringConverter

```

- `mq-sink.properties`: Create this file (ensure it's EBCDIC) to configure writing to MQ.

```
Properties
```

```
name=my-mq-sink-connector
connector.class=io.confluent.connect.jms.IbmMqSinkConnector  # Note different class name for sink
tasks.max=1 # Adjust as needed

# Kafka Source
topics=YOUR_KAFKA_SOURCE_TOPIC

# MQ Connection (Bindings Mode)
mq.queue.manager=YOUR_QMGR_NAME
# mq.hostname= (Leave blank or remove for bindings)
# mq.channel= (Leave blank or remove for bindings)
mq.transport.type=bindings
jms.destination.name=YOUR_MQ_DESTINATION_QUEUE_NAME
jms.destination.type=queue
mq.user.name=IF_NEEDED
mq.password=IF_NEEDED

```

```
# Converters (usually inherit from worker, but can override)
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.storage.StringConverter

```

4. **Launching Connect (Standalone Mode)**

Navigate to the `bin` directory within your Connect installation. Execute the script, providing the framework properties and one connector properties file:

```
Sh
```

```
# To run the MQ Source connector:
./connect-standalone ../etc/kafka/connect-standalone.properties ../etc/kafka/mq-source.properties

# To run the MQ Sink connector (in a separate terminal/process):
./connect-standalone ../etc/kafka/connect-standalone.properties ../etc/kafka/mq-sink.properties

```

5. **Distributed Mode**

The connect-standalone.properties settings will work in connect-distributed properties. With some additions as below.

```
# Kafka Cluster Connection (Update with your details)
bootstrap.servers=YOUR_KAFKA_BOOTSTRAP:9092
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="YOUR_KAFKA_API_KEY" password="YOUR_KAFKA_API_SECRET";
ssl.endpoint.identification.algorithm=https

# Producer settings (used by source connectors and internal Kafka Connect topics)
# Producer settings are often inherited from the worker configuration, but can be overridden here
# if needed for specific producer properties like a different client ID or acks.
# producer.sasl.jaas.config=... (if different from worker)
# producer.acks=all

# Consumer settings (used by sink connectors and internal Kafka Connect topics)
# Consumer settings are often inherited from the worker configuration, but can be overridden here
# if needed for specific consumer properties.
# consumer.sasl.jaas.config=... (if different from worker)

# Converters (adjust if using Avro, JSON Schema, etc.)
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.storage.StringConverter
# If using Avro, Schema Registry URL would be needed:
# key.converter.schema.registry.url=http://your-schema-registry:8081
# value.converter.schema.registry.url=http://your-schema-registry:8081

# ------------------------------------------------------------------------------
# Distributed Mode Specifics (Replaces standalone.properties offset.storage.file.filename)
```

```
# These topics must exist in your Kafka cluster and are used by Connect for coordination.
# Ensure the topics are configured for appropriate replication and retention.

# Offset Storage (shared across all Connect workers)
offset.storage.partitions=25 # Number of partitions for the offset topic
offset.storage.replication.factor=3 # Replication factor for the offset topic
offset.storage.topic=connect-offsets # Topic for storing connector offsets

# Configuration Storage (shared across all Connect workers for connector and task configurations)
config.storage.partitions=1 # Typically 1 partition is sufficient
config.storage.replication.factor=3 # Replication factor for the config topic
config.storage.topic=connect-configs # Topic for storing connector configurations

# Status Storage (shared across all Connect workers for connector and task statuses)
status.storage.partitions=5 # Number of partitions for the status topic
status.storage.replication.factor=3 # Replication factor for the status topic
status.storage.topic=connect-status # Topic for storing connector status updates

# Internal Kafka Connect Consumer Group ID
group.id=connect-cluster # Unique ID for this Connect cluster, used for internal consumer groups

# ------------------------------------------------------------------------------
# Plugin Path (Update with your actual paths on each worker)
plugin.path=/u/connectuser/connectors/mq-source,/u/connectuser/connectors/mq-sink

# REST API Settings (Each worker will expose its own REST API)
# The default port is 8082 for distrinuted. If running multiple Connect workers on the same host,
# you'd need to set unique ports for each.
rest.port=8082
rest.advertised.host.name=your-connect-worker-hostname # Important for REST API discovery by other workers
rest.advertised.port=8082 # Important for REST API discovery by other workers

# Worker ID (Each worker in the cluster needs a unique ID)
# It's good practice to set this explicitly, e.g., to the hostname or an instance ID.
# If not set, Connect will generate one, but explicit is better for monitoring.
# worker.id=connect-worker-1

# Error Handling (Optional, but recommended for production)
errors.tolerance=all # Allow errors in individual records without failing the entire task
errors.log.enable=true
errors.deadletterqueue.topic.name=my-connect-dlq # Topic for sending records that cause errors
errors.deadletterqueue.topic.replication.factor=3
errors.deadletterqueue.topic.partitions=1

# Loggers (Optional, typically configured in log4j.properties or logback.xml)
# log.level.root=INFO
# log.level.org.apache.kafka.connect=INFO
```

Navigate to the `bin` directory within your Connect installation. Execute the script, providing the framework properties and one connector properties file:

```
Sh
```

```
# To run in ditributed mode:
./connect-distributed ../etc/kafka/connect-distibuted.properties
```

Each connector will need to be added using the **Connect REST API**.

Here's the process to add a connector to the distributed cluster:

1. **Prepare Connector Configuration (JSON):**
   - Create a JSON object that defines the connector you want to add. This includes:
     - `name`: A unique name for this connector instance.
     - `config`: An object containing all the configuration properties specific to this connector (like `connector.class`, `tasks.max`, Kafka topic, MQ connection details, converters, etc.).
   - The structure is similar to the example shown on page 6 of the quickstart guide you provided.

   Example JSON structure for an MQ source connector (based on the guide's examples):

```
JSON
```

```
{
  "name": "my-mq-source-connector-01",
  "config": {
    "connector.class": "io.confluent.connect.ibm.mq.IbmMQSourceConnector",
    "tasks.max": "1",
    "kafka.topic": "YOUR_KAFKA_DESTINATION_TOPIC",
    "mq.queue.manager": "YOUR_QMGR_NAME",
    "mq.transport.type": "bindings",
    "jms.destination.name": "YOUR_MQ_SOURCE_QUEUE_NAME",
    "jms.destination.type": "queue",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.storage.StringConverter"
    // Add other necessary properties like security, transforms, etc.
  }
}

```

2. **Send HTTP POST Request:**
   - Use an HTTP client tool (like `curl` or Postman) or a script to send an HTTP `POST` request to the `/connectors` endpoint of one of your running Connect workers.
   - The default REST API port for distributed mode is 8082.
   - The request body must be the JSON configuration you created in step 1.
   - Set the `Content-Type` header to `application/json`.

Example using `curl` *(replace placeholders):*

```
Sh
```

```
curl -X POST -H "Content-Type: application/json" --data '{
  "name": "my-mq-source-connector-01",
  "config": {
    "connector.class": "io.confluent.connect.ibm.mq.IbmMQSourceConnector",
    "tasks.max": "1",
    "kafka.topic": "YOUR_KAFKA_DESTINATION_TOPIC",
    "mq.queue.manager": "YOUR_QMGR_NAME",
    "mq.transport.type": "bindings",
    "jms.destination.name": "YOUR_MQ_SOURCE_QUEUE_NAME",
    "jms.destination.type": "queue",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.storage.StringConverter"
  }
}' http://<your-connect-worker-hostname>:8082/connectors

```

Once you send this `POST` request, the Connect cluster will receive the configuration, create the connector instance, and distribute its tasks among the available workers. You can then use other REST API endpoints (like `GET /connectors`, `GET /connectors/{name}/status`, `PUT /connectors/{name}/config`, `DELETE /connectors/{name}`) to manage the connector's lifecycle.

General guidelines for making this a standard started proc on the mainframe.

The easiest way to do this is to create a wrapper script and reference that in your JCL.

A start-connect.sh file with the following inside it:

```
#!/usr/lpp/IBM/foz/v1r1/bin/bash
# Kafka Connect Distributed Starter Script

# Set environment
export LIBPATH=$LIBPATH:/usr/lpp/wsmq/mqm/V91R2103/java/lib
export CLASSPATH=/u/kafconn/confluent/confluent-connect-7.5.2-
zOS/plugins:/u/kafconn/confluent/confluent-connect-7.5.2-zOS/share/java

# Change to bin directory
cd /u/kafconn/confluent/confluent-connect-7.5.2-zOS/bin

# Start Kafka Connect
exec ./connect-distributed ../etc/kafka/connect-distributed.properties
```

OUTSIDE THAT SCRIPT:

`chmod +x /u/kafconn/confluent/start-connect.sh`

`mkdir -p /u/kafconn/logs`

Then reference that from the JCL:

```
//KAFKCONN PROC
//CONNECT EXEC PGM=BPXBATCH,REGION=0M,TIME=NOLIMIT
//STDPARM DD *
SH /u/kafconn/confluent/start-connect.sh
/*
//STDOUT DD PATH='/u/kafconn/logs/connect.out',
//       PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//       PATHMODE=SIRWXU
//STDERR DD PATH='/u/kafconn/logs/connect.err',
//       PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//       PATHMODE=SIRWXU
// PEND
```

1. **Dedicated script** - Like most z/OS started tasks use wrapper scripts
2. **Proper logging** - Redirects stdout/stderr to files instead of SYSOUT
3. **Environment isolation** - All environment setup contained in one place
4. **Easy to control** - Can be started/stopped cleanly via JCL
5. **Uses `exec`** - Replaces the shell process with the Java process (standard practice) – if you try and invoked the USS string of commands from JCL without this, you will run into ASCII vs EBCDIC issues.

To ensure your Java application, running under a specific RACF user, can trust a service using a Let's Encrypt certificate, you can use a **RACF keyring**. While the CA might be in the Java `cacerts` file, the standard and most secure practice on z/OS is to manage certificates centrally within RACF.

The process involves adding the Let's Encrypt root certificate to the RACF database and then connecting it to a keyring that the application's user ID has permission to read.

Using a RACF Keyring

Step 1: Add the Let's Encrypt Root CA to RACF

First, you need to ensure the Let's Encrypt root certificate (ISRG Root X1) is in the RACF database. You'll need to get the certificate file, upload it to a z/OS dataset, and then add it.

Use the following `RACDCERT` command. You only need to do this once.

```
RACDCERT CERTAUTH ADD('LETSENCRYPT.ISRG.ROOT.X1.CERT') +
  DSN('YOUR.PDS(CERTNAME)') +
  TRUST
```

```
4
```

- `LETSENCRYPT.ISRG.ROOT.X1.CERT` is a unique **label** you assign to the certificate within RACF.
- `DSN(...)` points to the dataset containing the certificate file.

Step 2: Create a Keyring for Your Application

If your application doesn't already have one, create a keyring owned by the RACF user ID that will run the application.

```
RACDCERT ADDRING(YourAppKeyring) ID(app_userid)

```

- `YourAppKeyring` is the name you choose for the keyring.
- `app_userid` is the RACF user ID that runs your Java application (e.g., your Kafka Connect started task user).

Step 3: Connect the CA Certificate to the Keyring

Link the trusted Let's Encrypt CA certificate to the new keyring.

```
RACDCERT CONNECT(CERTAUTH LABEL('LETSENCRYPT.ISRG.ROOT.X1.CERT') +
  RING(YourAppKeyring) USAGE(CERTAUTH)) ID(app_userid)

```

This command tells RACF that the certificate labeled `LETSENCRYPT.ISRG.ROOT.X1.CERT` should be treated as a trusted Certificate Authority for any application using `YourAppKeyring`.

Step 4: Configure Java to Use the RACF Keyring

Finally, you must tell the Java Virtual Machine (JVM) to use the RACF keyring as its truststore instead of the default `cacerts` file. This is done by setting Java system properties when you start your application.

Add these `-D` options to your Java startup command or configuration:

```
Java
```

```
-Djavax.net.ssl.trustStore=safkeyring://app_userid/YourAppKeyring
-Djavax.net.ssl.trustStoreType=JCERACFKS

```

- The `safkeyring://` URL tells Java to look for a keyring owned by `app_userid` named `YourAppKeyring`.

- `JCERACFKS` specifies that the keystore type is a RACF keyring.

This configuration directly instructs the Java process running as `app_userid` to "see" and use the trust information managed by RACF. So the above bash script becomes:

```
#!/usr/lpp/IBM/foz/v1r1/bin/bash
# Kafka Connect Distributed Starter Script

# Set environment
export LIBPATH=$LIBPATH:/usr/lpp/wsmq/mqm/V91R2103/java/lib
export CLASSPATH=/u/kafconn/confluent/confluent-connect-7.5.2-zOS/plugins:/u/kafconn/confluent/confluent-connect-7.5.2-zOS/share/java

# ---> ADD THIS LINE <---
# Configure Java to use the RACF keyring for SSL connections
# Replace 'app_userid' and 'YourAppKeyring' with your actual values
export KAFKA_OPTS="-Djavax.net.ssl.trustStore=safkeyring://app_userid/YourAppKeyring -Djavax.net.ssl.trustStoreType=JCERACFKS"

# Change to bin directory
cd /u/kafconn/confluent/confluent-connect-7.5.2-zOS/bin

# Start Kafka Connect
exec ./connect-distributed ../etc/kafka/connect-distributed.properties
```

OR you can create a JCL procedure (PROC) that uses the `BPXBATCH` utility.

`BPXBATCH` is a z/OS program that runs shell scripts and commands in a USS environment.

Here is a comprehensive example of how to set this up. Caveat: this will fallback to sh, so make sure teh shebang at the top of the connect-distributed and kafka-run-class references the full path to bash like #!/usr/lpp/IBM/foz/v1r1/bin/bash

---

Part 1: The JCL Procedure (PROC)

You will create a member in one of your system's PROCLIB concatenations (e.g., `SYS1.PROCLIB` or a user-defined PROCLIB). Let's name this member `KAFCONN`.

```
SAMPLE JCL
```

```
//KAFCONN PROC JUSER='KAFUSER',                    <-- USERID FOR THE STC
//            KAFHOME='/u/kafconn/confluent',           <-- KAFKA INSTALL DIR
```

```
//             KAFVER='confluent-connect-7.5.2-zOS',    <-- VERSION SUBDIR
//             CNFPROP='connect-distributed.properties', <-- CONFIG FILE
//             MQHOME='/usr/lpp/mqm/V9R2M0'             <-- IBM MQ INSTALL PATH
//*-------------------------------------------------------------------
//* KAFKA CONNECT DISTRIBUTED STARTED TASK FOR z/OS
//*
//* THIS PROC STARTS THE KAFKA CONNECT DISTRIBUTED WORKER
//* USING BPXBATCH TO RUN THE SHELL SCRIPT FROM USS.
//*
//* BASED ON CONFLUENT CONNECT ON Z/OS QUICKSTART GUIDE
//*-------------------------------------------------------------------
//CONNECT EXEC PGM=BPXBATCH,REGION=0M,TIME=NOLIMIT,
//        PARM='SH &KAFHOME./&KAFVER./bin/connect-distributed
&KAFHOME./&KAFVER./etc/kafka/&CNFPROP.'
//*
//*-------------------------------------------------------------------
//* ENVIRONMENT SETUP - CRITICAL FOR MQ INTEGRATION
//*-------------------------------------------------------------------
//STDENV   DD PATH='&KAFHOME./config/connect.env',PATHOPTS=ORDONLY
//*
//*-------------------------------------------------------------------
//* STEPLIB FOR IBM MQ NATIVE LIBRARIES (SCSQAUTH & SCSQANLE)
//* REQUIRED FOR BINDINGS MODE CONNECTION TO MQ
//*-------------------------------------------------------------------
//STEPLIB  DD DSN=CSQ.SCSQAUTH,DISP=SHR
//         DD DSN=CSQ.SCSQANLE,DISP=SHR
//*
//*-------------------------------------------------------------------
//* STANDARD I/O - CAPTURE OUTPUT TO JES SPOOL
//*-------------------------------------------------------------------
//STDOUT   DD SYSOUT=*
//STDERR   DD SYSOUT=*
//*
//*-------------------------------------------------------------------
//* STARTUP COMMAND:  S KAFCONN
//* SHUTDOWN COMMAND: P KAFCONN (GRACEFUL) or F KAFCONN,SHUTDOWN
//* CANCEL COMMAND:   C KAFCONN (FORCE STOP)
//*-------------------------------------------------------------------
//         PEND

```

Part 2: The Environment Variables File (`connect.env`)

The `STDENV` DD statement is the cleanest way to manage your environment variables. In your Kafka installation directory (e.g., `/u/kafka/confluent-7.6.1/config/`), create a file named `connect.env`.

This file should contain all the exports your startup script needs.

```
Bash
```

```
# ---------------------------------------------------------------------
# Environment variables for KAFCONN Started Task
# Based on Confluent Connect on z/OS Quickstart Guide
# ---------------------------------------------------------------------
```

```
5
6  # ------------------------------------------------------------------
7  # JAVA ENVIRONMENT
8  # ------------------------------------------------------------------
9  export JAVA_HOME=/usr/lpp/java/J11.0_64
10
11 # ------------------------------------------------------------------
12 # KAFKA/CONFLUENT ENVIRONMENT
13 # ------------------------------------------------------------------
14 export KAFKA_HOME=/u/kafconn/confluent/confluent-connect-7.5.2-zOS
15
16 # ------------------------------------------------------------------
17 # IBM MQ INTEGRATION - CRITICAL FOR BINDINGS MODE
18 # As specified in Confluent z/OS quickstart guide
19 # ------------------------------------------------------------------
20 export LIBPATH=$LIBPATH:/usr/lpp/mqm/V9R2M0/java/lib
21
22 # STEPLIB for MQ native libraries (alternative to JCL STEPLIB)
23 # Uncomment if not using JCL STEPLIB approach
24 # export STEPLIB=CSQ.SCSQAUTH:CSQ.SCSQANLE:$STEPLIB
25
26 # ------------------------------------------------------------------
27 # MQ JAVA LIBRARIES IN CLASSPATH
28 # Required for MQ connector operation - do not prepend with $CLASSPATH
29 # to avoid duplicates as warned in the guide
30 # ------------------------------------------------------------------
31 export
   CLASSPATH=/usr/lpp/mqm/V9R2M0/java/lib/com.ibm.mqjms.jar:/usr/lpp/mqm/V9R2M0/java/lib/com.ib
   m.mq.jmqi.jar:/usr/lpp/mqm/V9R2M0/java/lib/com.ibm.mq.allclient.jar
32
33 # ------------------------------------------------------------------
34 # KAFKA CONNECT PLUGIN PATH
35 # Points to MQ Source and Sink connector directories
36 # ------------------------------------------------------------------
37 export KAFKA_CONNECT_PLUGINS=/u/kafconn/connectors/mq-source,/u/kafconn/connectors/mq-sink
38
39 # ------------------------------------------------------------------
40 # KAFKA_OPTS: CRITICAL FOR z/OS PERFORMANCE
41 # Set heap size appropriate for your workload
42 # Enable zIIP offloading for better performance
43 # ------------------------------------------------------------------
44 export KAFKA_OPTS="-Xms4g -Xmx4g -Djava.awt.headless=true -XX:+UseG1GC -
   XX:+UseStringDeduplication"
45
46 # ------------------------------------------------------------------
47 # LOGGING CONFIGURATION
48 # ------------------------------------------------------------------
49 export LOG4J_CONFIGURATION_FILE=file:$KAFKA_HOME/etc/kafka/connect-log4j.properties
50
51 # ------------------------------------------------------------------
52 # z/OS SPECIFIC SETTINGS FOR BETTER COMPATIBILITY
53 # ------------------------------------------------------------------
54 export _BPXK_AUTOCVT=ON
55 export _CEE_RUNOPTS="ENVAR(JAVA_HOME=$JAVA_HOME)"
56
57 # ------------------------------------------------------------------
58 # ADDITIONAL ENVIRONMENT VARIABLES FOR MONITORING/DEBUGGING
59 # Uncomment as needed
```

```
# ------------------------------------------------------------------
# export KAFKA_DEBUG=true
# export KAFKA_JVM_PERFORMANCE_OPTS="-server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -
XX:InitiatingHeapOccupancyPercent=35"
```

Part 3: Explanation and Breakdown

1. **The `PROC` Statement**:
   - `//KAFCONN PROC ...`: Defines the start of a procedure named `KAFCONN`.
   - `JUSER='KAFUSER'`: A symbolic parameter for the User ID. This makes it easy to see and change.
   - `KAFHOME=...`: The root directory of your Confluent Platform or Kafka installation in USS.
   - `CNFPROP=...`: The name of your Connect properties file. This allows you to start the task with different configurations if needed.
2. **The `EXEC` Statement**:
   - `PGM=BPXBATCH`: This tells JES to run the `BPXBATCH` program.
   - `REGION=0M`: Gives the address space all the virtual storage it needs below and above the 2GB bar. This is standard practice for long-running Java applications on z/OS.
   - `TIME=NOLIMIT`: Prevents the started task from timing out.
   - `PARM='SH ...'`: This is the parameter passed to `BPXBATCH`. `SH` tells it to execute a shell command. The command is the full path to your `connect-distributed.sh` script followed by the path to its properties file. The symbolic variables (`&KAFHOME`, `&CNFPROP`) are substituted at startup.
3. **The `DD` Statements**:
   - `STDENV`: This is the most important part. `BPXBATCH` reads this file and applies all the `export` commands, setting up the perfect environment for the shell script to run, just as if you had sourced the file yourself.
   - `STDOUT` / `STDERR`: These capture the standard output and standard error streams from the script. Directing them to `SYSOUT=*` sends them to the JES spool, where you can view them like any other batch job output. This is where you will find startup messages, exceptions, and console logs.

Implementation Steps

1. **Customize the Files**:
   - Edit the `KAFCONN` JCL PROC to match your system's paths and conventions.
   - Create and edit the `connect.env` file in your USS directory, paying close attention to `JAVA_HOME` and setting an appropriate heap size (`-Xmx`).
2. **Place the JCL in PROCLIB**:
   - Save the `KAFCONN` JCL into a dataset that is part of your system's PROCLIB concatenation (e.g., `USER.PROCLIB` , `SYS1.PROCLIB`). Ask your z/OS Systems Programmer if you are unsure where to put it.
3. **Define Security (CRITICAL STEP)**:
   - A started task must be associated with a user ID. You need to define a rule in your security product (RACF, ACF2, or Top Secret).
   - **For RACF**, the commands would look something like this:

```
SAMPLE RACF
```

```
RDEFINE STARTED KAFCONN.* STDATA(USER(KAFUSER) GROUP(KAFGRP))
SETROPTS RACLIST(STARTED) REFRESH

```

   - The user ID (`KAFUSER` in this example) needs:
     - An OMVS segment.
     - `READ` and `EXECUTE` permissions on the `JAVA_HOME` and `KAFKA_HOME` directories and files in USS.
     - `WRITE` permission on any directories where logs or data are written.
4. **Start and Stop the Task**:
   - Once the JCL is in PROCLIB and the security is defined, you can use standard MVS console commands.
   - **Start:** `S KAFCONN`
   - **Stop (Graceful Shutdown):** `P KAFCONN` (This sends a SIGTERM signal, which the Kafka Connect script traps to perform a clean shutdown).
   - **Cancel (Force Stop):** `C KAFCONN`
5. **Check the Output**:

- Use SDSF or your job scheduler to look for the `KAFCONN` started task. The output in `STDOUT` and `STDERR` will tell you if it started correctly or if there were any errors.